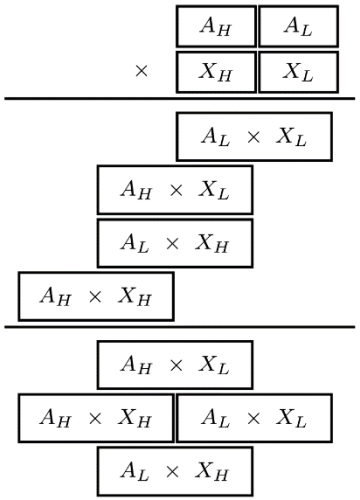
ALU HW3:
Multi-Precision FLP Dot-Product (DP)

Outlines

- FLP dot-product-4 (DP4) $z=x_1*y_1+x_2*y_2+x_3*y_3+x_4*y_4$
  - dual precision (32-b single-precision and 16-b half-precision)
  - shared computation resource
- 4-stage pipelining
  - unpack, fixed-point multiplication, alignment
  - fixed-point addition, conversion to sign-magnitude
  - normalization
  - rounding, pack
- clock gating for un-necessary computation units
  - some units not used in half-precision
  - power measurement using Prime Time

Dot Product (FLP_DP)

- dot-product of two 4-D vectors in IEEE FLP formats
  - z=x1*y1+x2*y2+x3*y3+x4*y4
  - 32-b single precision (SP) data (1,8,23)
    - ✓ unsigned Q1.23 multiplication
  - or 16-b half-precision (HP) data (1, 5, 10)
    - ✓ unsigned 1.10 multiplication
    - ✓ need one extra bit for precision mode selection
  - shared hardware with clock-gating



- sub-word parallelism
  - e.g., 24x24 multipliers could be constructed from four 12x12 multipliers
  - speed oriented
    - ✓ 4 times speed with half-precision
  - power-oriented
    - ✓ 3 unused 12x12 multipliers are turned off using clock-gating

Pipeline (FLP_DP_4)

- 1st stage
  - unpack (one extra input bit for SP/HP)
  - $ep_i=ex_i+ey_i$, i=1, 2, 3, 4
  - emax = max (ep_1, ep_2, ep_3, ep_4)
  - multiplication of $p_i= mx_i * my_i$, i=1, 2, 3, 4
  - alignment right-shift of non-max products
  - conversion to signed
- 2nd stage
  - fixed-point multi-operand addition
- 3rd stage
  - conversion to unsigned, LOD and normalization shift
- 4th stage
  - rounding, pack

comparison table

- table of summary

		Area (um²)			Delay	Latency	Power (W)		
		CL	SL	Total	(ns)	(ns)	dynamic	leakage	total
FLP_DP	area								
	mid								
	delay								
FLP_DP_4	area								
	mid								
	delay								

unsigned multiplication 24 x 24

- (sx_i, ex_i, mx_i), (sy_i, ey_i, my_i), i=1,2,3,4
- unsigned multiplication
  - $p_i = mx_i * my_i$ with unsigned Q1.23 (SP) or Q1.10 (HP)
  - 24*24 multiplier composed of four 12*12 (Q2.10 * Q2.10)
  - latch-gating for three pairs of (x_i, y_i) in case of HP
  - clock-gating for products in pipelined registers
- alignment for unsigned p_i -> p_i' with Q(2.50)
  - emax = max ($ep_i=ex_i+ey_i$, i=1,2,3,4)
  - fractional bits include 46 + 2 (rounding/guard) + 2 (four p_i)
  - consider only emax-ep_i <= 50+1=51 (p_i <4)
- conversion of unsigned Q2.50 p_i to signed Q(5.50)
  - signed_p_i' = (sx_i*sy_i == 1) ? $signed($p_i$') : $signed(-$p_i$'+1);
  - Q3.50 for | signed_p_i' | < 4
  - Q5.50 for addition of four signed_p_i', i=1, 2, 3, 4
- Q5.24 for each signed roduct in half precision ?

Single-/Half-Precision (SP/HP)

- signle preicision
  - unsigned multiplication Q1.23 * Q1.23 = Q2.46
    - ✓ 24*24 multipliers
  - alignment right shift => Q2.50 (?)
    - ✓ consider exponent difference <= 51
  - conversion to signed for each product => Q3.50
  - addition of four signed proudcts => Q5.50
    - ✓ 55-bit signed adder
- hald precision
  - unsigned multiplication Q1.10 * Q1.10 = Q2.20
    - ✓ 12*12 multipliers (shared with above 24*24 multiplier)
  - alignment right shift => Q2.24 (?)
    - ✓ consider exponent difference <= 25
  - conversion to signed for each product => Q3.24
  - addition of four signed proudcts => Q5.24
    - ✓ 29-bit signed adder (shared with above 55-bit adder)

Clock Gating DC Synthesis Scripts

- auto clock-gating (not recommended)
  - use DC command “compile -gate_clock”

```
always @ (posedge clk)
   if (en) q <= d;
```

```
compile –gate_clock
```

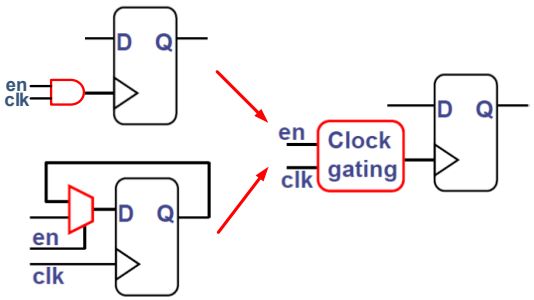
or

```
insert_clock_gating
compile
```

- manual clock-gating

```
assign gclk = clk & en;
always @ (posedge gclk)
   q <= d;
```

```
replace_clock_gates
compile
```



- manual clock-gating 2

```
assign gclk1 = clk & en1;
assign gclk2 = clk & en2;
always @ (posedge gclk1) q1 <= d1;
always @ (posedge gclk2) q2 <= d2;
```

```
replace_clock_gates -global
compile
```

- report clock gating

```
report_clock_gating  -gating_elements
```

- set constraints for clock gating

```
set_ideal_network [get_pins pin_name
```

latch-gated vs. clock-gated multiplier

- input-latched multiplier
  - keep multiplier inputs unchanged in case of unused conditions
  - multiplier inputs are from gated latches
- input-clock-gated multiplier
  - multiplier inputs are from clock-controlled FF
- output-clock-gated multiplier
  - multiplier output is passed to clock-gated FF
  - multiplier still consumes power due to input changes
  - but new multiplier output is NOT stored in the clock-gated FF

```
// input-latched multiplier

wire [15:0] x, y;
reg [15:0] xtmp, ytmp;
wire [31:0] prod;

always @ (en or x or y)
   if (en) begin
      xtmp = x;
      ytmp = y;
   end
assign prod = xtmp * ytmp;
         // prod not stored
```

```
// input-clock-gated multiplier
reg  [15:0] x, y;  reg [31:0] prod;

assign gclk = clk & en;
always @ (posedge gclk) begin xtmp<=x;  ytmp<=y; end
assign prod = xtmp * ytmp;  // prod not stored
```

```
// output-clock-gated multiplier
wire [15:0] x, y;  reg [31:0] prod;

assign gclk = clk & en;
always @ (posedgelk) prod <= x * y;  // x, y not stored
```